

Cluster and Cloud Computing Assignment 2 - Big Data Analytics on the Cloud

Background

In development and delivery of non-trivial software systems, working as part of a team is generally (typically!) the norm. This assignment is very much a group project. Students will be put into software teams to work on the implementation of the system described below. These will be teams of up to 5 students. In this assignment, students need to organize their team and their collective involvement throughout. There is no team leader as such, but teams may decide to set up processes for agreeing on the work and who does what. Understanding the dependencies between individual efforts and their successful integration is key to the success of the work and for software engineering projects more generally. If teams have “issues”, then please let me know asap and I will help resolve them.

Assignment Description

The software engineering activity builds on the technologies taught in the lectures and demonstrated in the workshops including the MRC/NeCTAR Research Cloud, Kubernetes, Docker, Fission and ElasticSearch. The project focuses on big data analytics using a range of social media data sets including one or more of BlueSky, GDELT, Mastodon and Reddit. Teams should come up with one or more scenarios that combine and analyse these data sets. These scenarios should be connected in some way to Australia noting that these social media providers and the posts they contain do not explicitly contain location information, e.g. a lat/long of where someone posted. Rather students should explore less precise location information, e.g. GDELT news items related to Australia, subreddits that have a connection with Australia (such as r/Melbourne), Mastodon servers related to Australia (such as AusSocial, Mastodon Australia – see <https://mastodonservers.net/>), BlueSky accounts/posts potentially related in some way to Australia, e.g. AustralianLabour, GuardianAustralia, 6NewsAustralia.

Students may decide to explore different world events as the basis for their scenarios, e.g. tariffs, wars, elections that are reported in mainstream media. Students are encouraged to explore the Australian Internet Observatory (<https://www.aio.eresearch.unimelb.edu.au/>) to determine if data exists for the scenarios of interest.

Teams need to develop clients that harvest/stream data from one or more of the aforementioned social media platforms and include them in the ElasticSearch database using Fission and Kubernetes. Students may also use the Mastodon/BlueSky data from assignment 1 that is downloadable from SPARTAN. Teams may decide to obtain multiple API keys to collect data from these platforms.

External Data Providers

- **BlueSky** – for details on accessing and using the BlueSky API please see <https://docs.bsky.app/>.
- **GDELT** – for details on accessing and using GDELT please see <https://www.gdeltproject.org/data.html>.
- **Mastodon** – for details on accessing and using the Mastodon API please see <https://docs.joinmastodon.org/client/intro/>.
- **Reddit** – for details on accessing and using the Reddit API please see <https://www.reddit.com/r/reddit.com/wiki/api/>.

Teams are expected to develop a range of analytic scenarios comparing harvested/streamed external data. Teams are free to explore any scenarios that connect “in some way” to Australia. Teams are encouraged to be creative here. For example, teams may look at scenarios such as:

- What do Australian's think of politicians in Australian and indeed globally, e.g. Anthony Albanese, Angus Taylor, Donald Trump, Keir Starmer, Emmanuel Macron, Vladimir Putin or Xi Jinping across the different social media platforms?
- What do Australians think of the global conflicts, e.g. the wars in Ukraine, Iran, or what is happening in Gaza? Is one platform more vocal (have more posts) on these events and is the sentiment of the population changing?
- Which AFL team has the most followers and has the highest/lowest sentiment and how does that change with the win/loss ratio and league position?
- Are people happier when the weather is good?
- What foods/drinks are mentioned the most in social media and which ones do people like the most?
- Does the stock market change with public sentiment, e.g. if sentiment on Telstra goes up then does the price of Telstra go up on the ASX (<https://www.marketindex.com.au/asx200>)?
- Who are the major "influencers" on the different platforms (most followers, most posts)?

The above are examples – students may decide to create their own analytics based on the data they obtain. Students are not expected to build advanced “general purpose” data analytic services that can support any scenario but show how tools like ElasticSearch with targeted data analysis capabilities can be used to capture the essence of life in Australia and how public opinion changes. **A prize will be awarded for the most interesting scenarios identified!**

The front-end to the system should be a JupyterNotebook.

For the implementation, teams are recommended to use a commonly understood language across team members – most likely Python. Teams are free to use any pre-existing software systems that they deem appropriate for the analysis and visualisation, e.g., NLTK, Vader, TextBlob and existing topic models, e.g. LDA, BERT.

Students should also include at least one scenario using other more official data. These can be obtained from SUDO (<https://staging.unimelb-sudo.cloud.edu.au/>), from Government open data platforms such as (<https://www.data.vic.gov.au/>) or from specific organisations, e.g. weather data from the Australian Bureau of Meteorology (<https://www.bom.gov.au/>) or others.

Error Handling

Issues and challenges in using the NeCTAR Research Cloud for this assignment should be documented. You should describe the limitations of mining content and language processing (e.g., sarcasm). You should outline any solutions developed to tackle such scenarios.

Final packaging and delivery

You should collectively write a team report on the application developed and include the architecture, the system design and the discussions that lead into the design. You should describe the role of the team members in the delivery of the system and where the team worked well and where issues arose and how they were addressed. The team should illustrate the functionality of the system through a range of scenarios and explain why you chose the specific examples. Teams are encouraged to write this report in the style of a paper that can ultimately be submitted to a conference/journal. If teams use LLMs then this should be documented and details provided in how and where they were used.

Each team member is also expected to complete a confidential report on their role in the project and their experiences in working with their individual team members. This will be handed in separately to the final team report. (This is not to be used to blame people, but to ensure that all team members are able to provide feedback and to ensure that no team has any member that does nothing!).

The length of the team report is not fixed. Given the level of complexity of the assignment and total value of the assignment, a suitable estimate is a report in the range of 20-25 pages. A typical report will comprise:

- A description of the system functionalities, the scenarios supported and why, together with graphical results, e.g., charts/graphs related to the scenarios;
- A discussion on the pros and cons of the NeCTAR Research Cloud and tools and processes for delivery of the platform, e.g. challenges with Kubernetes, Fission or ElasticSearch;
- Demonstrating how the application can dynamically scale, e.g. adding new social media harvesters dynamically to collect more data from the platforms;
- Teams should also produce a video of their system demonstrating its core functionality. This video should be uploaded to YouTube (these videos can last longer than the Cloud deployments unfortunately!) and the link for this included in the report;
- Reports should also include a link to the source code (please use gitlab - <https://gitlab.unimelb.edu.au>).
- The code repository should be named "*COMP90024_team_<team number>*".
- A README must be provided in the code repository including the installation procedure (excluding the standard setup covered in the workshop). This should cover instructions on how to run the application, instructions on how to test the system, and a description of the codebase layout (folders and subfolders).

It is recommended that students follow the code repository structure identified here:

README.md: a document that contains a brief description of the repo contents, installation instructions, and instructions on how to use the client
frontend: source code of the client part of the application (Jupyter notebook)
backend: the application back-end source code (harvesters, analytics, APIs etc)
test: the application back-end automated testing of the source code
database: ElasticSearch type mappings, queries, etc.
data: Any data you want to put in the code repository
docs: Documentation (your report in PDF format, and any information needed to understand your code etc)
llm (optional): If you used an LLM in the assignment, you should document your use, e.g. identify the models/prompts used, and any/all documentation in how you used the LLM.

Also to note:

- The repositories must be set to private. You should add the teaching team to your repository (git account details will be shared in due course). The teaching team should be added with *developer* or *maintainer* roles.
- All teams should share their *kubeconfig* files with the teaching team when they have a stable cluster established. The cluster should be called comp90024. The kubeconfig file should adopt the following naming convention: *config.<zero padded team number>.yaml*, e.g. *config.04.yaml* for team #4.
- Extraneous files, e.g. *.idea*, **.pyc*, *_pycache*, *node_modules* etc should not be included in the repository. The repositories should also be cleaned up generally, e.g. removing unnecessary Fission resources, ElasticSearch Indexes and removal of *queries* from the database folder structure.

It is important to put your collective team details (team number, names, surnames, student ids) in:

- the head page of the report;
- as a header in each of the files of the software project.

Individual reports describing your role and your teams' contributions should be submitted through a link that will be sent through in due course.

Implementation Requirements

Teams are expected to use:

- a version-control system (gitlab) for sharing source code.
- Use of Fission for stream data processing and data ingestion.

- Use of ElasticSearch for data management and analytics.
- Use of Kubernetes.
- The server side of your application should expose its data to the JupyterNotebook through a ReSTful design. Authentication or authorization is NOT required for the front end.
- Rich analytics and visualisation of data through the JupyterNotebook.

Teams are also encouraged to describe:

- How fault-tolerant is your software setup? Is there a single point-of-failure?
- Support for integrated software testing during application development.
- Can your application and infrastructure dynamically scale out to meet demand?

Deadline

One copy of the team assignment is to be submitted through Canvas. The zip file must be named with your team, i.e. <CCC2026-TeamN>.zip.

Individual reports describing your role and individual team member contributions should be submitted a link that will be distributed in due course. These individual reports will be the completion of web-based forms, i.e., they do not require Word/PDF documents etc.

The deadline for submitting the team assignment is **Wednesday 20th May (by 12 noon!)**. Note that this is a hard deadline as we are at the end of the course and the demonstrations commence the day after!

Marking

The marking process will be structured by evaluating whether the assignment (application + report) is compliant with the specification given. This implies the following:

- A working demonstration of the Cloud-based solution and use of Kubernetes – **25% marks**
- A working demonstration of stream data harvesting using Fission and use of ElasticSearch for specific data collection and analytics scenarios – **25% marks**
- Detailed documentation on the scenarios, the system architecture and the design – **20%**
- Report and write up discussion including pros and cons of the NeCTAR Research Cloud and the use of Kubernetes, Fission and ElasticSearch for big data analytics – **20% marks**
- Proper handling of errors – **10% marks**

The (confidential) assessment by your peers in your team will be used to weight your individual scores accordingly. Timeliness in submitting the assignment in the proper format is important. **A 10% deduction per day will be made for late submissions.**

Demonstration Schedule and Venue

The student teams are required to give a presentation (with a few slides) and a demonstration of the working application. The presentation should include the key data analytics scenarios supported as well as the design and implementation choices made. Each team has **up to 15 minutes** to present their work. This will take place on:

- **9-11am 21st May (8 teams present – randomly selected from teams 1-45)**
- **9-11am 22nd May (8 teams present – randomly selected from teams 45-90)**

Note that given the numbers of teams this year, not all teams will be able to present – however all teams should be prepared to present. I will advise on Canvas how the randomised selection process will be arranged.

As a team, you are free to develop your system(s) where you are more comfortable with (at home, on your PC/laptop, or in the labs...) but obviously the demonstration should work on the NeCTAR Research Cloud.